#SKILL - 8 Spring Boot – JPQL & Query Methods Module

Prerequisites:

- Basic Idea on Spring Boot and Spring Data JPA
- Knowledge of writing derived query methods

An e-commerce application needs a product search feature where users can view products by category, filter items within a price range, and sort products by price. The backend team must implement these operations using Spring Data JPA, including derived query methods and JPQL queries, to fetch data efficiently without writing long SQL statements.

Your task is to build a product search module that supports category-based search, price filtering, and sorting using Spring Data JPA.

Tasks:

1. Create a Product entity with the following fields: a. id
   b. name
   c. category
   d. price

```java
package com.ecommerce.product.model;

import jakarta.persistence.Entity;

@Entity
public class Product {

    @Id
    private int id;
    private String name;
    private String category;
    private double price;

    public Product() {}

    public Product(int id, String name, String category, double price) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.price = price;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }
```

2. Create a ProductRepository and add derived query methods:
   a. findByCategory(String category)
   b. findByPriceBetween(double min, double max)

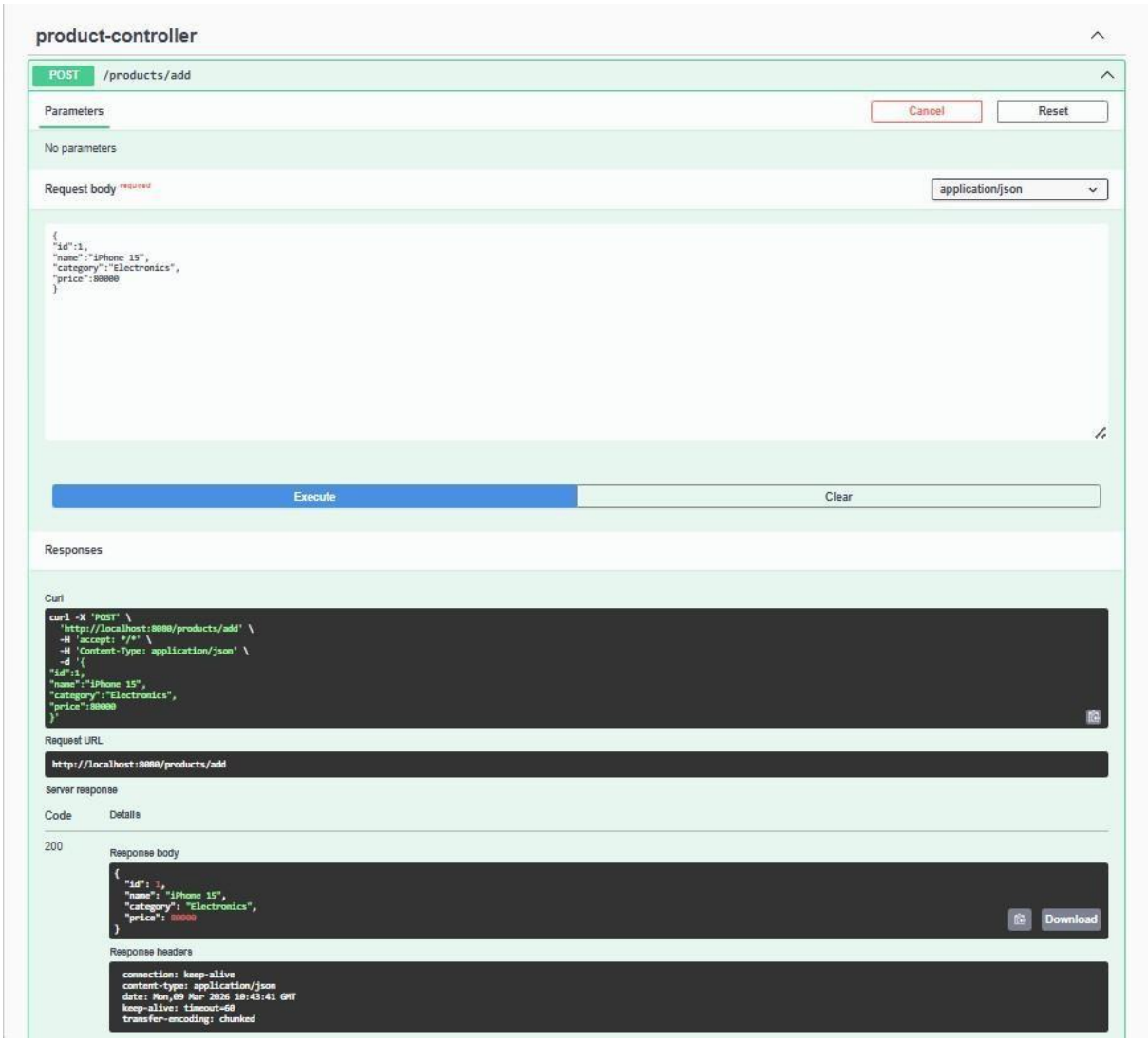
```java
package com.ecommerce.product.repository;

import java.util.List;

public interface ProductRepository extends JpaRepository<Product, Integer> {

    List<Product> findByCategory(String category);

    List<Product> findByPriceBetween(double min, double max);

    @Query("SELECT p FROM Product p ORDER BY p.price ASC")
    List<Product> getProductsSortedByPrice();

    @Query("SELECT p FROM Product p WHERE p.price > ?1 price ")
    List<Product> getExpensiveProducts(double price);

    @Query("SELECT p FROM Product p WHERE p.category = ?1 category ")
    List<Product> getProductsByCategoryJPQL(String category);
}
```

3. Write JPQL queries using @Query for:
   a. Sorting products by price
   b. Fetching products above a price value
   c. Fetching products by category

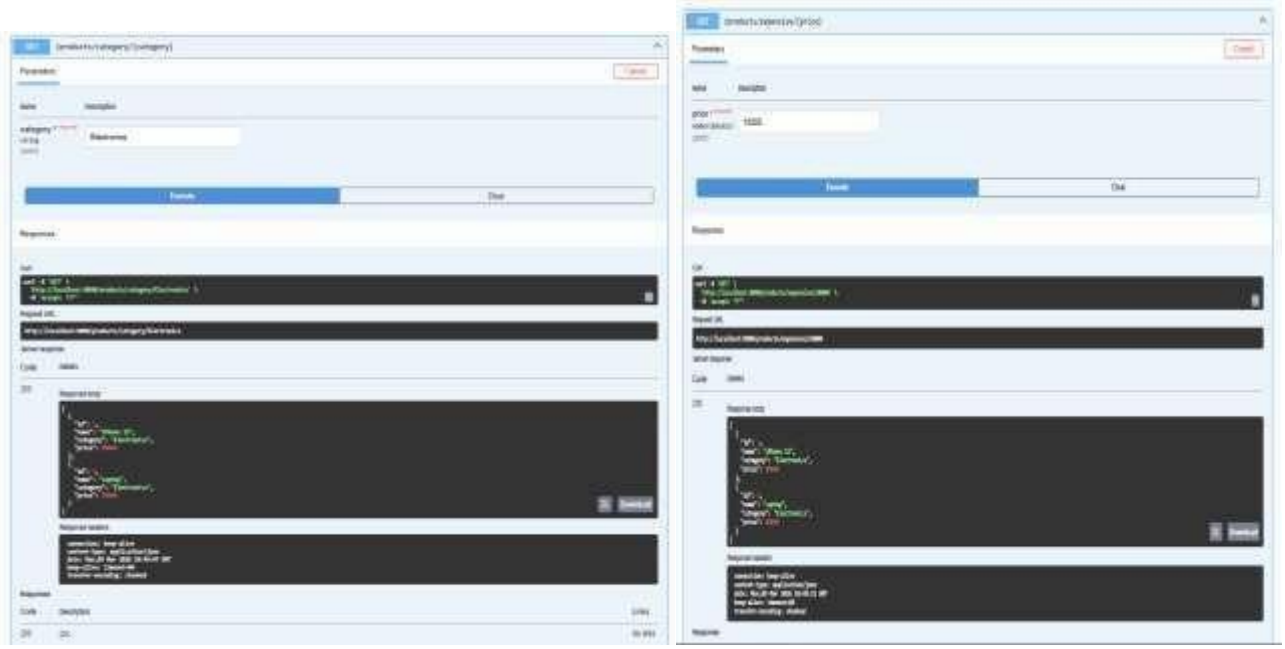
4. Create REST controller endpoints to test query methods:
   a. /products/category/{category}
   b. /products/filter?min=&max=
   c. /products/sorted
   d. /products/expensive/{price}

```java
package com.ecommerce.product.controller;

import java.util.List;

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductRepository repo;

    @PostMapping("/add")
    public Product addProduct(@RequestBody Product product) {
        return repo.save(product);
    }

    @GetMapping("/category/{category}")
    public List<Product> getByCategory(@PathVariable String category) {
        return repo.findByCategory(category);
    }

    @GetMapping("/filter")
    public List<Product> filterPrice(@RequestParam double min, @RequestParam double max) {
        return repo.findByPriceBetween(min, max);
    }

    @GetMapping("/sorted")
    public List<Product> sortByPrice() {
        return repo.getProductsSortedByPrice();
    }

    @GetMapping("/expensive/{price}")
    public List<Product> expensiveProducts(@PathVariable double price) {
        return repo.getExpensiveProducts(price);
    }
}
```

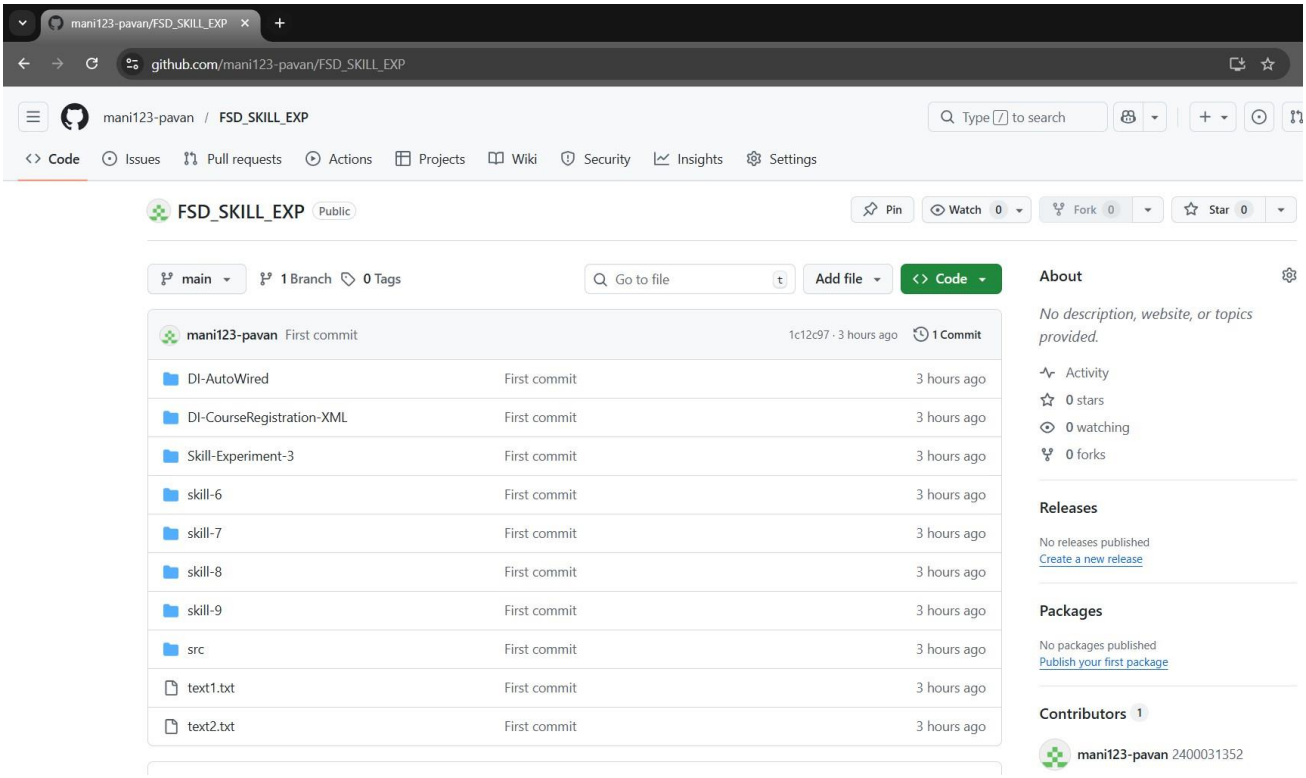
5. Insert sample product records and test using Postman.



6. Verify output for different categories, ranges, and price values.



7. Push the Spring Boot project to GitHub.

8. 

GitHub Link : https://github.com/Anil1843/fsad-skill

VIVA QUESTIONS:

1. What is JPQL and how is it different from SQL?

JPQL (Java Persistence Query Language) is a query language used to query **entity objects in JPA**, while SQL queries **database tables directly**.

2. What are derived query methods in Spring Data JPA?

Derived query methods are queries automatically generated by Spring Data JPA **based on the method name**.

Example:

findByCategory(String category)

3. Why is method naming important in Spring Data JPA?

Spring Data JPA **parses the method name** to automatically generate the correct database query.

4. Can JPQL return custom objects in a query?

Yes. JPQL can return **custom DTO objects** using constructor expressions.

Example:

SELECT new com.example.ProductDTO(p.name,p.price) FROM Product p

5. When should we choose derived queries instead of JPQL?

Derived queries should be used when the query is **simple and can be expressed using method names**, while JPQL is used for **complex queries and custom logic**.

(For Evaluator's use only)

Comment of the Evaluator (if Any)

Evaluator's Observation

Marks Secured:_____out of _____

EMP ID & Full Name of the Evaluator:

Signature of the Evaluator Date of Evaluation: